HUMANIFYOS · ENGINEERING TALK

Spec coding with Cursor

What 600+ agent-built tickets taught me about turning specs into software

Shawn Lewis
Chief Technology Architect, TTEC HumanifyOS

October 2026

THE SHIFT

The spec is the product. Agents compile it into code.

HUMANIFYOS PACK

600+

tickets across 3 programs (platform, admin, end-user UI); 487 planned in one spec pack

TRADITIONAL ESTIMATE

41.3k h

18–24 months for a 12–16 person team, about $5.12M

ACTUAL, CURSOR AGENTS 24/7

10 days

from first ticket to built, against a 17-day plan

The work moved upstream. Most of my time went into the spec, the tickets and the rules, not into reviewing code line by line.

THE PLAYBOOK

Nine practices that made it work

01
Prep: three inputs into one technical PRD, iterated

02
One prompt that finalizes the technical requirements

03
A Jira ticket template, enforced in code

04
Skills layered on your requirements and grounded in the spec

05
Model selection by ticket size

06
Unit testing over and above normal

07
Take nothing for granted: test every click

08
Automate 100%: scripts, installs, Terraform

09
Docs updated with every ticket

Three inputs become one technical PRD

Business and product
Plain-English functions, features, personas

Example screens
HTML mocks, videos, screenshots

Technical requirements
Stack locks, flows, data, integrations



ONE TECHNICAL PRD

Broken into components and systems, then iterated many times until complete

- Requirements ledger: every item gets an ID and a source
- Gap report: contradictions, stubs, missing pieces
- Freeze file: decisions locked, assumptions written down

In HumanifyOS: a 181-row ledger and a 60-row gap report for the LOOM data module alone, a decisions log with rulings numbered past Q-400, and pack versions v1.0 to v3.4.

The spec kept growing during the build

PLANNED

487

tickets in the first Jira import



BUILT

600+

tickets by the end, in 10 days

120+ tickets nobody planned

- Holes: things the spec never defined
- Updates: things the spec defined wrong
- Bug fixes: things that did not work

Budget about 25% for discovered work, and put every new ticket through the same template and generators, so the spec stays the source of truth.

/build-pack: the prompt behind every spec folder

build-pack/SKILL.md · opening

```
You are the principal architect and delivery
lead for the project. Turn everything available
(the conversation, every attached document and
pack, the newest material) into ONE final build
pack, better than any of its inputs and
complete enough that a coding harness that
knows nothing can build the whole system 0→1.

Do not ask clarifying questions unless a
decision is irreversible. Take the most
reasonable reading, record it as an assumption,
keep going.
```

Its non-negotiable rules

1. **Read everything, in full.** Skimming is a failure.
2. **Nothing is lost.** Every requirement gets a ledger row.
3. **Newest wins for stack; the union wins for features.**
4. **Interrogate before you merge.**
5. **The harness knows nothing.** Exact names beat descriptions.
6. **Validate with code, not confidence.**
7. **Design for the fastest build:** contracts first, folder locks.

Nine phases, from scattered inputs to a validated pack

0	1	2	3	4	5	6	7	8
Intake	Gaps	Freeze	Spec	Preamble	Tickets	Plan + Jira	Validate	Deliver

The spec pack a ticket may cite

PRODUCT · FUNCTION-CATALOG · MONOREPO · PIPELINE · SCHEMA + sql/ · openapi/*.yaml · EVENTS · CONFIG-KEYS · AUTH-TENANCY · SCREENS · FIXTURES · INSTALL · DOCS-IN-CODE · OBSERVABILITY

A ticket may only reference what a spec file defines.

Generators produce the tickets, then validation checks the schema, cycles, folder-lock conflicts and every link before anything ships.

One bar for every ticket

Field	The rule
Summary	80 characters or fewer, prefixed with the plan ID
Why	40–150 words on what breaks if it is not built
What	60+ words; numbered deliverables with exact paths, tables, routes, events
Acceptance criteria	3+ Given/When/Then with real data; 1+ negative or edge; exactly one “Test command:”
Lock units	The folders this agent owns; two running agents never share one
Technical details	API contract, migration, config keys, idempotency, vendors
UI	Screen file, route, states, accessibility, visual baseline
Blocked by	Minimal and true; becomes a Jira Blocks link
Links	Every spec file it relies on; each must exist

common.T(...) asserts the bar: a ticket that misses it fails the build, so it never reaches Jira.

Q-45: the Emerging issues section

Story · APP · ui · High · blocked by Q-00 Q-02 Q-04 Q-61 Q-16 A-53

What: deliver apps/web/quality/src/modules/voc/emerging/ + src/api/app-voc-emerging.ts; card click goes to /drill/vol/topic/<name>; live frame hos.voc.emerging.v1 refetches.

AC-1 (happy) Given cx_emerging.json, the pill reads “3 rising”; clicking the first card navigates to its drill route.

AC-3 (negative) On a 503 the section falls back to the overview list with a “Live update unavailable” note and one retry, no retry storm.

AC-5 (visual) Render at 1440×900 matches cx-emerging.png within 5%.

AC-6 Test command: pnpm --filter quality test -- voc/emerging

Why it builds right first time

- Exact folders it owns
- The screen file is the design of record
- Clicks and failures are ACs
- Fixture data, not a live backend
- One command proves it
- 12 links back to the spec

Blockers turn tickets into a schedule

487
issues in the first Jira import

1,664
Blocks links, each one checked

18
agents at once, merge on green

0
duplicate external IDs

Platform starts
T+0

→

Admin starts
G-READY green ≈ T+167h

→

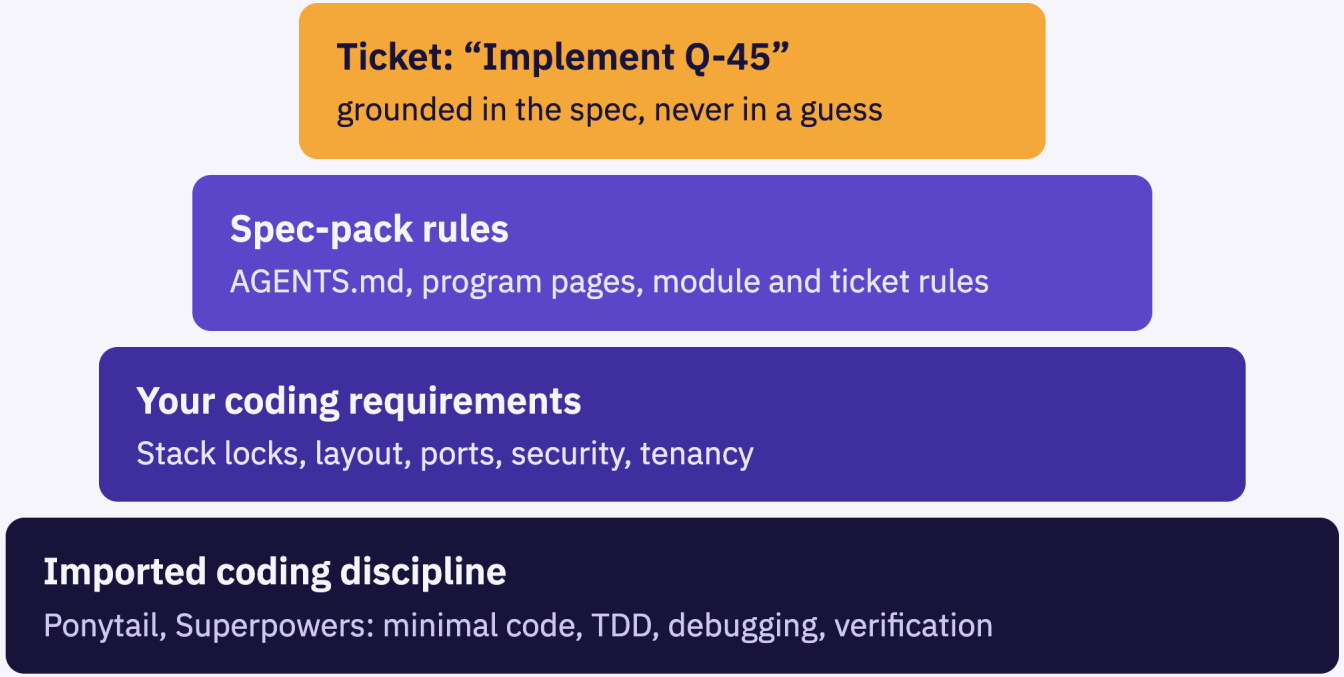
UI starts
G-RBAC green ≈ T+225h

→

All done
plan 17 days, actual 10

Jira Cloud import rules: external ID = plan ID · status “To Do” · strip reporter, created and Story Points (a locked field in Jira Cloud).

Build skills on top of the requirements you already have



Every ticket, screen and function traces back to a spec file that lives in the repo.

Cursor cloud agents only see skills and rules committed to the repo, in .cursor/skills and .cursor/rules. Plugins on someone’s laptop never reach them.

Five skill packs worth importing

Skill pack	Why import it	Stars · license
Ponytail DietrichGebert/ponytail	Stops over-building: reuse, stdlib or native first, then minimum code. Keeps validation, security and tests on risky logic.	~178k · MIT
Superpowers obra/superpowers	TDD, systematic debugging, verification before “done”, plans, and a spec-compliance review before the code-quality review	~305k · MIT
Spec Kit github/spec-kit	Spec workflow: constitution, specify, plan, tasks, implement. OpenSpec (~73k, MIT) is lighter for existing code.	~144k · MIT
Agent Skills addyosmani/agent-skills	Review gates: code review, security hardening (OWASP, secrets), doubt-driven review; /ship runs four reviewers in parallel	~110k · MIT
Anthropic skills anthropics/skills	skill-creator: the template for writing your own spec-pack skills	~183k · mostly Apache-2.0

Commit vendored copies to .cursor/skills/, pinned to a commit, refreshed quarterly.

One pack folder, three programs, read-only to agents

```
HOSPACK/
├─ HOS/ platform: LOOM · SWATCH · RUBRIC · CORE
│  ├─ LOOM/spec/ PRODUCT · SCHEMA · openapi · EVENTS
│  ├─ LOOM/ REQUIREMENTS-LEDGER · GAP-REPORT · ui/
│  ├─ tickets/ jira/ plan/ generated
│  ├─ repo-root/ AGENTS.md · CLAUDE.md · .cursor/
│  ├─ tools/ generators · validate.py · regen_all.sh
│  └─ JIRA_IMPORT_ALL.json · KICKOFF.md
├─ HOS-Admin/ AdminServer + admin UI
└─ HOS-UI/ HOSAppServer + TTEC Quality UI
```

Two zones

The pack is read-only. Code, live contracts and docs live at the repo root of HumanifyOS (Go) and HumanifyOS-UI (TypeScript).

.cursor/rules

00-hospack-always
02-program-*
10-frontend · 20–24 modules
30–35 topics · 40-ticket
45-ui-clicks

One small rule is always on; module and topic rules load by folder.

Warp runs the pack: one parent, up to 18 agents

`/warp-init` → `/warp-scan HOSPACK/HOS` → `/warp-start G0-G3`

Warp
The parent: scans, dispatches, and is the only merger

Shuttle
One ticket, one branch, one pull request

Reed
Bugbot, evidence, auto-merge or hold

Herald
Slack and Teams: status, alarms, approvals

Jira moves with the work

To Do → In Progress → In Review (Bugbot) → QA Ready (L, XL) → Done

No sledgehammer for simple UI work

Size	Hours	Shuttle model	Merge
S	4	claude-haiku-5-5	Auto
M	7	claude-sonnet-5-5-high	Auto
L	11	claude-sonnet-5-5-high	Waits in QA Ready
XL	16	claude-opus-5-5	Waits in QA Ready
Fix	—	claude-opus-5-5	From fix round 2

“Auto” means after Bugbot is clean, CI is green and every acceptance criterion passes. A ticket that fails 5 fix rounds parks for a person.

```
.warp/config.yaml
model: claude-sonnet-5-5-high
modelS: claude-haiku-5-5
modelXL: claude-opus-5-5
fixModel: claude-opus-5-5
fixModelAfter: 2
maxFixAttempts: 5
autoMergeSizes: S, M
maxAgents: 18
```

Tests are how the agent proves the ticket

One test per AC

Named for it: Test_Q45_AC3_..., so review is a checklist

One command

Every ticket names the command that must exit 0 before the PR opens

Negative and edge

At least one per ticket: 403s, 503s, empty data, wrong tenant

Fixtures, not vendors

Recorded data and mock handlers; no live services in unit tests

Visual baseline

The render matches the screen PNG within 5%

Accessibility

axe on every route and modal state, zero violations

On top of that, gates run real end-to-end checks: make ready and make e2e, in a 12-minute budget.

If it is clickable, it has a test

```
.cursor/rules/45-ui-clicks.mdc
---
description: "Every click is a test case"
globs: "apps/web/**/*.tsx"
---
On every page, any element whose click opens
a popup, navigates or runs a function is a
test-case requirement:
1. One test per clickable: render, click,
assert the visible result.
2. A removed or broken handler fails a test.
3. The PR lists every clickable and its test.
```

Agents leave handlers unwired, and later edits to a page quietly break them.

Q-45 · AC-1

“...clicking it navigates to the drill route for that topic”

The test outlives the ticket, so the next change cannot break the click silently.

If a person has to do it by hand, it is a ticket

Command	What it replaces
make infra-up	Start stores, Dapr, migrations, seed and readiness in one target
scripts/hos-ready.sh	A person reading twelve logs; writes one ready.json
seed-org.sh	Hand-built test tenants, programs and LOBs
hos-token mint	Hand-made JWTs for tests and agents
bootstrap.sh	Copying rules and contracts into each repo
regen_all.sh	Editing tickets, Jira files and plans by hand
Terraform / compose	Console clicks; the same install runs in CI

The goal is 100% automation: an agent at 3 a.m. can stand up, prove and reset the stack alone.

Docs ship in the same PR, or the ticket is not done

`docs/technical/`

Technical

Architecture per module, generated flag tables

`docs/user/`

User

Screens and how-tos per app

`docs/runbooks/`

Operational

Replays, restores, secret rotation

`docs/adr/`

Decisions

One record for every locked choice

DOCS-IN-CODE: merge-blocking

- Every package has a comment naming its callers, calls and events
- Every handler lists its route, scope, schemas and error codes
- Every config key and flag is documented, or spec-lint fails the PR

TAKEAWAYS

Write the spec so well the agent never has to ask

1 Iterate the PRD, then budget ~25% for what the build finds

2 Generate tickets and enforce the template in code

3 Commit skills and rules to the repo, grounded in the spec

4 Size the work, match the model, test every AC and every click

5 Automate everything; docs ship with the code

Start Monday: run /build-pack on one module and import Ponytail and Superpowers.